

Security Drill

社内セキュリティ訓練アプリ

「引っかかって、気づいて、強くなる」

セキュリティ意識を体験から変える

なぜ作ったか

社長なりすましで Chatwork にメッセージが届く——そんな事例が現実には起きている。
AI が普及した今、セキュリティへの危機感は確実に高まっている。

なのに、肌感覚でセキュリティを学べる場がない。

- e-learning や資料配布 → 読んで終わり、行動は変わらない
- 「気をつけましょう」と言うだけでは、人は変わらない
- 訓練が形骸化している現場を何度も見てきた

→ 実際に体験しなければ、「気づき」は定着しない

Security Drill が目指すもの

本番に近い訓練メールを実際に送り、引っかかったその場で学んでもらう

1. 管理者が AI でフィッシングメールを生成・配信
2. 受信者がリンクをクリック → 「引っかかった」と記録
3. その場で学習ページ＋セキュリティクイズへ誘導
4. 合格するまで繰り返し → スコアで組織のリテラシーを可視化

組織の「気づき」を一過性で終わらせず、定着させる 仕組み

もうひとつの狙い — 社内開発の活性化

システム会社として、**自分たちで作って自分たちで使う** 実績を作りたい。

- 社内開発アプリを実運用している事例がまだ少ない
- 自社プロダクトの開発・運用経験は **技術力向上に直結** する
- セキュリティ訓練は全社員に関わるテーマ → 社内展開しやすい

社内ツールから始めて、社内開発の文化を育てる取り組みにしたい

管理者の流れ

訓練の作成から配信まで、**一つの画面で完結**。

- シナリオ選択（パスワード再設定・請求・配送 など）
- **AIでメール文面を生成**（件名・本文・誘導文）→ 手動編集も可能
- クイズも AI で一括生成 → 編集・保存
- 対象者・チャネル（メール / Slack）を選んで配信

利用者の流れ

届いたメールのリンクをクリックすると —

- クリックが記録され、学習ページへ誘導
- 危険性の説明を読む
- セキュリティクイズに合格するまで受験
- LLM が受験履歴を分析し、個別フィードバック

引っかかった人だけが「学ぶ」設計。

技術スタック

レイヤ	技術
フロント / バックエンド	Next.js (App Router) + TypeScript
DB	PostgreSQL + Prisma ORM
認証	Auth.js v5 (Google / LINE OAuth)
AI 生成	LangChain + Ollama (ローカル LLM)
通知	メール (SMTP) / Slack (Webhook)
インフラ	Docker / Podman でローカル完結

ポイント: ローカル LLM で社内データを外部に出さない安心設計

設計のこだわり — 過去の反省から

過去の失敗: ロジックとデザインの分離が甘く、バグを量産した

今回の設計判断:

- **クリーンアーキテクチャをフロントエンドまで一貫導入**
 - domain / usecase / infrastructure / UI をきっちり分離
 - 将来 NestJS 等へのバックエンド切り出しもスムーズ
- **Next.js フルスタックへの挑戦**
 - Next.js だけでどこまでいけるか試したかった
 - Async React の思想に寄せた SC / SA 前提の構成
 - 今後のバージョンアップや保守で楽になる方向

AI 活用とセキュリティ対策

プロンプト設計とインジェクション対策を仕様段階から組み込んだ

プロンプトインジェクション **多層防御**:

1. **静的チェック** — 危険パターンを正規表現で即座にブロック
2. **LLM ジャッジ** — 入力が「スタイル指定のみ」かを意味論的に判定
3. **プロンプト堅牢化** — ユーザー入力を隔離ブロックに閉じ込め
4. **出力の事後チェック** — 禁止表現の検出、JSON スキーマ検証

AI 駆動開発も試験的に導入

- 仕様書・ガイドを整備し、AI と人が同じ文脈で開発できる体制に

課題

- **ローカル LLM の生成品質** — 「訓練っぽさ」からの脱却が最優先。まず **RAG / Few-shot・プロンプト強化**、余力があれば **LoRA 等のチューニング** や **モデル差し替え** も、といったラインでざっくり検討中
- **時間的制約による簡略化** — 期限に合わせ、設計をスモールに留めた部分がある
- **AI 駆動開発のハーネス** — Cursor の制御・静的解析・SpecID ベースの検証など、AI 出力のミスを **どう炙り出すか** の設計がまだ固まっていない
- **シナリオ管理** — CRUD とプロンプトテンプレート管理（月1回程度の更新を見据えて）
- **ランダム配信** — BullMQ + Redis でジョブ化、状態管理とリトライ・冪等性
- **Slack 強化** — Web API による ID 正当性チェック、Bot での柔軟な配信
- **クイズ拡張** — 記述式（text）出題対応、問題数 10～15 問への拡張

まとめ

Security Drill は、

「送る → 引っかかる → 学ぶ」を回す **実践型セキュリティ訓練アプリ**

- 生成 AI 時代のセキュリティ課題に、**体験** で向き合う
- ローカル LLM × クリーンアーキで、**安全性と拡張性** を両立
- **自分たちで作って自分たちで使う**、社内開発の活性化にも繋げたい

セキュリティは「知っている」から **「体験して身につける」** へ。

ローカル LLM と安心設計、拡張設計で、そのまま実運用を想定した構成にしている。

ご清聴ありがとうございました！